docker run Command

Introduction

The docker run command is one of the most commonly used **Docker commands**. It is used to **create and start a container** from a Docker image.

- **What does docker run do?**
- **How does it work?**
- **What options can be used with docker run?**

This guide provides a **detailed explanation of docker run with examples**.

Section 1: Learn

What is docker run?

The docker run command **creates a container from an image and starts it immediately**.

Component	Description
Image	The base template used to create a container
Container	An isolated environment where the application runs
Run Flags	Additional options to customize the container behavior

How Does docker run Work?

When you execute docker run, Docker does the following:

1. **Checks if the image is available locally** (if not, it downloads from Docker Hub).
2. **Creates a new container** from the image.
3. **Allocates system resources** (CPU, memory, network).
4. **Executes the default command** inside the container.
5. **Keeps the container running (if applicable)**.

Interesting Fact

Each time you run docker run, a **new container** is created. If you don't **explicitly remove it**, the stopped container remains in the system.

Section 2: Practice

1. Running a Simple Container

```
docker run ubuntu echo "Hello, Docker!"
```

What happens here?

- Pulls the **Ubuntu image** (if not available locally).
- Creates a new container.
- Runs the echo command inside the container.
- **Container exits after execution**.

---

2. Running a Container in Interactive Mode

```
docker run -it ubuntu bash
```

What happens here?

- -i keeps the container's standard input open.

- -t allocates a terminal interface.
- Runs the bash shell inside the Ubuntu container.

To **exit the container**, type:

```
exit
```

---

3. Running a Container in Detached Mode

```
docker run -d nginx
```

What happens here?

- -d runs the container in the **background**.
- Starts an **Nginx web server**.

To check running containers:

```
docker ps
```

To stop the container:

```
docker stop <container_id>
```

---

4. Running a Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

What happens here?

- -p 8080:80 maps **port 8080 on the host** to **port 80 in the container**.
- Now, the Nginx web server is accessible at:

```
http://localhost:8080
```

---

5. Running a Container with a Custom Name

```
docker run --name my_nginx -d nginx
```

To verify:

```
docker ps
```

The container appears with the name **my_nginx**.

---

6. Running a Container with Volume Mounting

```
docker run -d -v /home/user/data:/app/data ubuntu
```

- -v mounts the **host directory** /home/user/data to the **container directory** /app/data.
- Any changes in /home/user/data are **reflected inside the container**.

---

7. Automatically Removing a Container After It Stops

```
docker run --rm ubuntu echo "This container will be removed"
```

- The container **runs and removes itself** after execution.

Section 3: Know More

Frequently Asked Questions

1. **How do I run a container interactively?**

```
docker run -it ubuntu bash
```

2. **How do I run a container in the background?**

```
docker run -d nginx
```

3. **What happens if I don't use** --rm**?**
    - The container **stays in a stopped state** and must be **manually removed**.
4. **How do I list all containers?**

```
docker ps -a
```

5. **How do I stop and remove a running container?**

```
docker stop <container_id>
docker rm <container_id>
```

Conclusion

The docker run command is **essential for starting and managing containers**. Start by **running containers in interactive mode, using background execution, port mapping, and volume mounting**, and soon you'll be **deploying applications with Docker easily!**